

TECMILENIO

Ingeniería de Software

Alimenta+

Sistema web para la gestión de donaciones de alimentos y recursos entre empresas y organizaciones sociales

Entrega final del reto

Implementación y seguridad · Pruebas y calidad · Cierre y evaluación

Integrantes

Mauricio Nuñez Pulido

Roberto Naredo Medellín

Johan Nuñez Pulido

Fecha: 27 de septiembre de 2026

Repositorio: <https://github.com/Rob-nm/alimenta-plus>

Contenido

1. Resumen ejecutivo
2. Implementación del módulo y seguridad
3. Pipeline de integración y entrega continuas
4. Pruebas de seguridad y análisis de calidad
5. Comparación entre lo planificado y lo ejecutado
6. Lecciones aprendidas
7. Plan de mejora continua
8. Conclusiones

Anexo A. Contenido del repositorio y evidencias

1. Resumen ejecutivo

En esta etapa final se implementó el primer incremento funcional de Alimenta+: el **microservicio de Usuarios y Autenticación**, que permite el registro de personas donantes, el inicio de sesión con **JSON Web Tokens (JWT)** y el control de acceso por **roles (administrador / usuario)**. Alrededor de este módulo se construyó un **pipeline de integración y entrega continuas en GitHub Actions** que, en cada cambio, ejecuta el lint y las pruebas, analiza la calidad con **SonarQube**, construye y publica una imagen Docker, la despliega en un entorno de prueba y la somete a un escaneo de seguridad con **OWASP ZAP**.

Los resultados finales son: **85 pruebas automatizadas** con **99.53 % de cobertura de sentencias** (umbral exigido: 80 %), **Quality Gate aprobado** en SonarQube con calificación **A** en seguridad, confiabilidad y mantenibilidad, **0 code smells** y **0 min de deuda técnica**, y un escaneo de OWASP ZAP con **0 alertas altas, 0 medias y 0 bajas**. Las 3 alertas de riesgo bajo y los 2 code smells detectados en la primera corrida completa se **corrigieron** y se verificaron con una segunda corrida del mismo pipeline.

Frente al plan original de 15 semanas, el proyecto adelantó las actividades de calidad y seguridad (planeadas para el sprint 4) y redujo el alcance de desarrollo a un microservicio de los cinco diseñados. El detalle de estas desviaciones, las lecciones aprendidas y el plan de mejora continua se presentan en las secciones 5, 6 y 7.

Indicador	Meta	Resultado final	Estado
Cobertura de pruebas unitarias	≥ 80 %	99.53 % sentencias · 98.87 % ramas	Cumplida
Autenticación JWT + roles	admin / usuario	JWT HS256 con expiración, issuer y audience; rutas protegidas por rol	Cumplida
Pipeline CI/CD automatizado	Pruebas, build y despliegue	4 etapas encadenadas; despliegue automático a staging en cada push a main	Cumplida
Vulnerabilidades (OWASP ZAP)	0 altas / medias	0 altas · 0 medias · 0 bajas	Cumplida
Quality Gate (SonarQube)	Aprobado	Aprobado · deuda técnica 0 min	Cumplida

2. Implementación del módulo y seguridad

2.1 Alcance del módulo

Se eligió el dominio de **Usuarios/Autenticación** de la arquitectura de microservicios propuesta en el avance, porque es la base de todos los demás módulos (donaciones, solicitudes y reportes dependen de saber quién es el usuario y qué puede hacer). El servicio está construido con **Node.js 22**, **Express 5** y **PostgreSQL 16**, y se entrega como imagen Docker.

Método	Ruta	Acceso	Descripción
POST	/api/auth/register	Público	Registro de persona donante (siempre con rol usuario)
POST	/api/auth/login	Público	Valida credenciales y emite un JWT de 1 hora
GET	/api/auth/me	Autenticado	Perfil del usuario del token
GET	/api/users	admin	Lista paginada de usuarios
GET	/api/users/:id	admin	Detalle de un usuario
PATCH	/api/users/:id/rol	admin	Cambia el rol (admin / usuario)

Método	Ruta	Acceso	Descripción
PATCH	/api/users/:id/estado	admin	Activa o desactiva una cuenta
GET	/health · /api-docs/openapi.json	Público	Salud del servicio y especificación OpenAPI 3

Tabla 1. Endpoints del microservicio de autenticación.

El código sigue una separación por capas: **rutas** (HTTP) → **middleware** (autenticación, autorización, validación, errores) → **servicio** (reglas de negocio) → **repositorio** (SQL). Esta separación permite probar cada capa de forma aislada y reemplazar la base de datos real por una en memoria durante las pruebas.

2.2 Autenticación con JWT y roles

- **Contraseñas** con bcrypt (10 rondas en producción); nunca se devuelven en ninguna respuesta. La contraseña exige 8 a 72 caracteres con mayúscula, minúscula y número.
- **Token JWT** firmado con HS256, con expiración de 1 hora y verificación estricta de algoritmo, emisor (issuer) y audiencia (audience). Se rechazan tokens con firma alterada, sin firma (alg: none) o expirados.
- **Roles:** el registro público siempre asigna el rol usuario, aunque el cliente intente enviar rol: admin. Solo un administrador puede promover a otro usuario, y ningún administrador puede quitarse permisos ni desactivarse a sí mismo.
- **Cuentas desactivadas** no pueden iniciar sesión (403).
- **Mensaje genérico y tiempo constante** en el login: se compara contra un hash ficticio cuando el correo no existe, para no revelar qué correos están registrados.
- El **administrador inicial** se crea al arrancar a partir de variables de entorno; no existen credenciales en el código.

2.3 Defensas contra inyección y XSS

- **Consultas SQL parametrizadas** (\$1, \$2...) en todo el repositorio de datos: ninguna entrada del usuario se concatena en el SQL.
- **Validación con listas blancas** (zod) en cuerpo, parámetros y query string: nombres, ciudad y organización solo aceptan letras, números y signos de puntuación básicos, por lo que se rechazan cargas como <script> o comillas de inyección. Los objetos son estrictos: se rechaza cualquier campo no esperado.
- Las respuestas son siempre JSON; el cuerpo de las peticiones se limita a 10 KB.

2.4 Pruebas unitarias

Las pruebas se escribieron con **Jest** y **Supertest**, usando **pg-mem** (PostgreSQL en memoria) para que las pruebas de integración ejecuten el SQL real sin depender de un servidor. El umbral de cobertura del 80 % está configurado en Jest, por lo que **el pipeline falla automáticamente si la cobertura baja**. En la versión final hay **85 pruebas** en 7 archivos:

Archivo	Qué valida
unit/authService.test.js	Registro, hash de contraseña, login, cuentas desactivadas, verificación de JWT, creación del admin
unit/middleware.test.js	authenticate, authorize, validate y manejador de errores

Archivo	Qué valida
unit/config.test.js	Variables de entorno obligatorias (JWT_SECRET) y valores por defecto
integration/auth.routes.test.js	Registro, login y perfil de punta a punta; JSON inválido; cuerpo demasiado grande
integration/users.routes.test.js	Control de acceso por rol, paginación, cambio de rol y estado, ids inválidos
integration/security.test.js	Inyección SQL, XSS y manipulación de JWT (firma alterada, alg none, expirado)
integration/hardening.test.js	Cabeceras de seguridad, CORS restringido y límite de intentos (429)

Tabla 2. Organización de las pruebas automatizadas.

Métrica de cobertura	Corrida #2 (antes)	Corrida #3 (final)
Sentencias	99.48 %	99.53 %
Ramas	98.82 %	98.87 %
Funciones	97.43 %	97.67 %
Líneas	99.45 %	99.5 %
Número de pruebas	75	85

Tabla 3. Cobertura reportada por Jest en el pipeline (reports/*/pruebas-unitarias).

3. Pipeline de integración y entrega continuas

El pipeline está definido en `.github/workflows/ci-cd.yml` y se ejecuta en cada push y pull request a la rama main. Las etapas están encadenadas: si una falla, las siguientes no se ejecutan, y todos los reportes se publican como artefactos descargables de cada corrida.

Etapas	Qué hace	Criterio de aprobación
1. Lint y pruebas	Instala dependencias, ejecuta ESLint y Jest con cobertura; publica cobertura HTML/lcov y JUnit	Sin errores de lint, todas las pruebas en verde y cobertura ≥ 80 %
2. SonarQube	Levanta SonarQube Community en el runner, analiza el código con la cobertura, exporta métricas, issues y capturas del tablero	Quality Gate = OK
3. Construcción	Construye la imagen Docker (usuario sin privilegios, healthcheck) y la publica en GitHub Container Registry con la etiqueta del commit y staging	Imagen construida y publicada
4. Despliegue + DAST	Despliega la imagen publicada con PostgreSQL (docker compose, environment staging), corre pruebas de humo y un escaneo autenticado de OWASP ZAP contra la API	Pruebas de humo OK y 0 alertas de riesgo alto

Tabla 4. Etapas del pipeline de CI/CD.

Los secretos del entorno de prueba (JWT_SECRET y la contraseña del administrador) se generan aleatoriamente en cada despliegue y se enmascaran en los registros, por lo que no se almacena ninguna credencial en el repositorio.

3.1 Historial de ejecuciones

Corrida	Commit	Resultado	Duración	Observación
#1	04b7ed3	Fallida	3 min 35 s	Lint, pruebas (99.48 % de cobertura) y SonarQube aprobados. La construcción de la imagen falló: npm ci exige package-lock.json. El pipeline bloqueó el despliegue.
#2	0fdf2b3	Exitosa	5 min 1 s	Primera corrida completa (línea base "antes"): imagen publicada, despliegue en staging, pruebas de humo OK. ZAP: 3 alertas bajas. SonarQube: 2 code smells.
#3	4e1e642	Exitosa	5 min 9 s	Corrida "después" de las correcciones: 85 pruebas, ZAP sin alertas bajas/medias/altas, SonarQube con 0 code smells y 0 min de deuda técnica.

Tabla 5. Ejecuciones del pipeline en GitHub Actions.

La primera corrida falló en la etapa de construcción porque el Dockerfile usaba **npm ci**, que exige un package-lock.json que no se había versionado. El pipeline detuvo la entrega antes de desplegar, que es exactamente su propósito; el problema se corrigió en un commit y la corrida #2 completó las cuatro etapas.

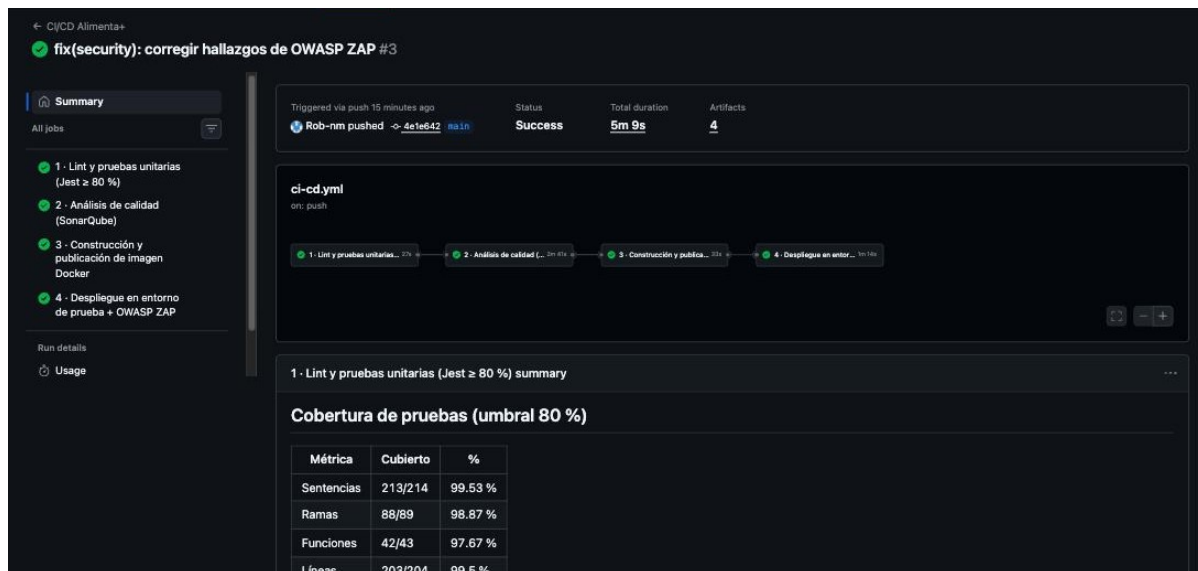


Figura 1. Ejecución final del pipeline en GitHub Actions.

4. Pruebas de seguridad y análisis de calidad

4.1 Escaneo de seguridad con OWASP ZAP

Se utilizó el **API scan de OWASP ZAP 2.17.0**, que importa la especificación OpenAPI del servicio y ejecuta un escaneo pasivo y **activo** (inyección SQL, XSS, path traversal, inyección de comandos, entre otros) contra el servicio desplegado en el entorno de prueba. El escaneo se autentica con un token de administrador para poder alcanzar también las rutas protegidas.

Riesgo	Corrida #2 (antes)	Corrida #3 (después)
Alto	0	0
Medio	0	0

Riesgo	Corrida #2 (antes)	Corrida #3 (después)
Bajo	3	0
Informativo	4	3

Tabla 6. Tipos de alerta de OWASP ZAP por nivel de riesgo.

El escaneo activo **no encontró inyección SQL ni XSS**, lo que confirma las defensas de la sección 2.3. Sí identificó tres hallazgos de riesgo bajo, que se corrigieron:

Hallazgo (ZAP)	CWE	Causa	Corrección aplicada	Estado
Server Leaks Information via "X-Powered-By" (10037)	497	Express agrega por defecto la cabecera X-Powered-By: Express y revela la tecnología del servidor	app.disable('x-powered-by')	Corregido
X-Content-Type-Options Header Missing (10021)	693	La primera versión no enviaba cabeceras de seguridad	helmet: nosniff, Content-Security-Policy, HSTS, frame-ancestors; Cache-Control: no-store	Corregido
A Server Error response code was returned (100000) — HTTP 500	388	Un id mayor al rango INTEGER de PostgreSQL pasaba la validación y la consulta fallaba	Límite de rango (2,147,483,647) en id y offset + prueba automatizada	Corregido

Tabla 7. Hallazgos de OWASP ZAP y su corrección.

El hallazgo más relevante fue el **error 500**: ZAP envió un identificador enorme (4656420115663864681) a /api/users/:id; el valor pasaba la validación y PostgreSQL rechazaba la consulta por estar fuera del rango del tipo INTEGER. Además de exponer un error interno, era una forma sencilla de generar errores en el servidor. Se añadió el límite de rango en la validación y una prueba automatizada que lo cubre.

De forma preventiva se agregaron también: **Content-Security-Policy** restrictiva, **Strict-Transport-Security**, **CORS** limitado a los orígenes configurados, **límite de intentos** en login y registro (respuesta 429) contra ataques de fuerza bruta, y se eliminó el secreto JWT por defecto que existía en la configuración inicial (ahora el servicio no arranca sin un JWT_SECRET de al menos 32 caracteres).

4.2 Análisis de calidad con SonarQube

El pipeline levanta un servidor **SonarQube Community** dentro del runner, analiza el código fuente junto con el reporte de cobertura de Jest y exporta las métricas mediante su API. Los resultados de ambas corridas son:

Métrica	Corrida #2 (antes)	Corrida #3 (después)
Quality Gate	Aprobado (OK)	Aprobado (OK)
Líneas de código	397	439
Cobertura	98.9 %	99.0 %
Líneas duplicadas	0.0 %	0.0 %
Bugs	0	0
Vulnerabilidades	0	0
Security hotspots	0	0
Code smells	2	0

Métrica	Corrida #2 (antes)	Corrida #3 (después)
Deuda técnica	10 min	0 min
Ratio de deuda técnica	0.1 %	0.0 %
Calificación de mantenibilidad	A	A
Calificación de confiabilidad	A	A
Calificación de seguridad	A	A
Complejidad ciclomática	87	93
Complejidad cognitiva	30	31

Tabla 8. Métricas de SonarQube (reports/*/sonarqube/sonar-measures.json).

Regla	Severidad	Archivo	Descripción	Corrección
javascript:S7721	Mayor	users.routes.js	La función assertNotSelf estaba anidada dentro de otra sin necesitar su contexto	Se movió al ámbito del módulo
javascript:S1481	Menor	authService.js	Variable _omit declarada solo para descartar password_hash	Se copia el objeto y se elimina la propiedad explícitamente

Tabla 9. Code smells detectados y corregidos.

La deuda técnica bajó de **10 min a 0 min** y el proyecto mantiene **0 % de duplicación, 0 bugs y 0 vulnerabilidades**. Cabe señalar una limitación: la edición Community de SonarQube no incluye el análisis de flujo de datos para inyecciones (taint analysis), por lo que la detección de SQLi y XSS se cubrió con las pruebas de seguridad de Jest y con el escaneo activo de OWASP ZAP.

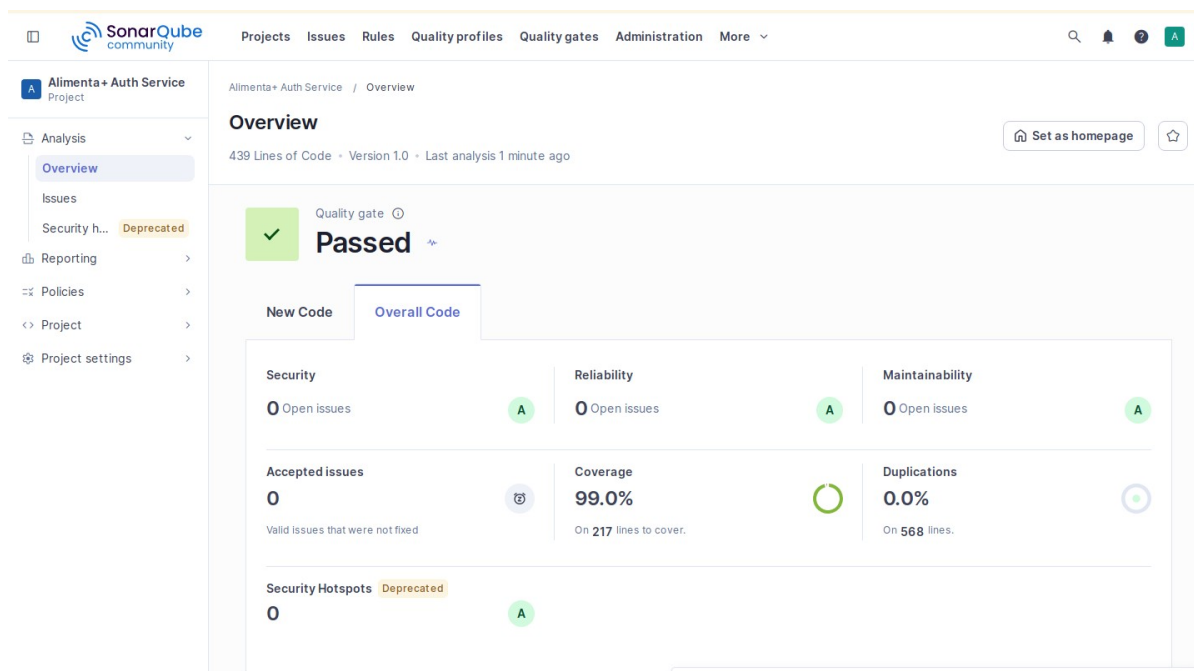


Figura 2. Tablero de SonarQube de la corrida final (Quality Gate aprobado).

5. Comparación entre lo planificado y lo ejecutado

La línea base es el cronograma del avance del reto (30 de agosto de 2026): tres fases de definición, diseño y planificación (semanas 1 a 6), cuatro sprints de desarrollo de dos semanas (semanas 7 a 14) y el lanzamiento del MVP en la semana 15. La entrega final ocurre el 27 de septiembre, aproximadamente en la **semana 10** del plan.

Actividad	Planificado	Ejecutado	Desviación y causa
Fase 1: Definición del proyecto	Semanas 1-2	Completada (acta de constitución)	Sin desviación.
Fase 2: Requerimientos y diseño	Semanas 3-5	Completada y entregada en el avance (30 de agosto)	Sin desviación. En la implementación se ajustó la capa de datos a PostgreSQL.
Fase 3: Riesgos y recursos	Semana 6	Completada	Sin desviación.
Sprint 1: Interfaz de usuario	Semanas 7-8	No se desarrolló la UI. Se construyó el microservicio de usuarios y autenticación (API REST)	Cambio de alcance: la etapa final exigía un módulo con JWT, roles y cobertura $\geq 80\%$, y todos los módulos dependen de la autenticación.
Sprint 2: Módulo de donaciones	Semanas 9-10	No iniciado	Pasa al backlog. Ya puede apoyarse en el servicio de usuarios terminado.
Sprint 3: Geolocalización y notificaciones	Semanas 11-12	No iniciado	Pasa al backlog; el riesgo R-01 (API de mapas) sigue vigente.
Sprint 4: Reportes, QA e integración	Semanas 13-14	QA e integración continua adelantadas a la semana 10 (pipeline, SonarQube, OWASP ZAP). Reportes pendientes	Desviación positiva en calidad: se aplica desde el primer incremento y no al final.
Hito: lanzamiento del MVP	Semana 15	Se entregó un incremento desplegable (auth-service en staging), no el MVP completo	El MVP se reprograma en la hoja de ruta (sección 7.3).

Tabla 10. Cronograma planificado contra ejecutado.

5.1 Alcance técnico planificado contra implementado

Elemento	Planificado (avance)	Implementado	Comentario
Arquitectura	5 microservicios + API Gateway + RabbitMQ	1 microservicio (Usuarios/Autenticación)	Se priorizó un servicio completo, probado y desplegable sobre cinco incompletos
Base de datos	PostgreSQL + MongoDB/PostGIS	PostgreSQL 16	PostGIS se incorporará con el servicio de geolocalización
Autenticación	Centralizada en el API Gateway	JWT emitido por el servicio de usuarios	El Gateway podrá validar el mismo token sin cambios en el servicio
Pruebas	Sprint 4 (QA)	Desde el primer commit, 80 % obligatorio	Se adelantó la calidad (shift-left)

Elemento	Planificado (avance)	Implementado	Comentario
Seguridad (RNF-03)	Cifrado y auditorías periódicas	bcrypt, JWT, cabeceras, ZAP en cada push	El cifrado en reposo queda pendiente para producción
Despliegue	No detallado en el avance	Imagen Docker en GHCR + staging automático	Nuevo entregable, no contemplado en el plan
Historias de usuario	5 historias (donaciones, mapa, notificaciones, seguimiento, reportes)	Registro y autenticación (prerrequisito de todas)	Las 5 historias pasan al backlog de los siguientes sprints

Tabla 11. Alcance técnico planificado contra implementado.

5.2 Registro de incidencias durante la implementación

#	Incidencia	Impacto	Resolución
1	La construcción de la imagen Docker falló en la corrida #1: npm ci requiere package-lock.json, que no estaba versionado	Se bloqueó la entrega de ese commit; retraso de aproximadamente 15 minutos	Se cambió a npm install en el Dockerfile y en el pipeline. Acción pendiente: versionar el lockfile (plan de mejora #1)
2	OWASP ZAP provocó un error 500 enviando un id fuera del rango de PostgreSQL	Error de robustez no cubierto por las pruebas iniciales	Validación de rango en id y offset + prueba automatizada
3	La primera versión no enviaba cabeceras de seguridad y exponía X-Powered-By; la configuración tenía un secreto JWT por defecto	3 alertas de riesgo bajo en ZAP; riesgo de despliegue con un secreto conocido	helmet, Cache-Control, CORS restringido, rate limiting y JWT_SECRET obligatorio
4	GitHub marcó la acción de SonarScanner v5 como vulnerable	Riesgo en la cadena de suministro del pipeline	Actualización a sonarqube-scan-action v6
5	Avisos de deprecación de Node.js 20 en varias acciones de GitHub	Sin impacto actual; podría romper el pipeline en el futuro	Registrado en el plan de mejora (#2)
6	SonarQube Community no incluye análisis de inyecciones (taint analysis)	SQLi y XSS no se detectan en el análisis estático	Se cubrieron con pruebas de seguridad en Jest y con el escaneo activo de ZAP

Tabla 12. Incidencias registradas en la bitácora de ejecución.

6. Lecciones aprendidas

6.1 Durante la planificación

El alcance técnico era demasiado amplio para el equipo y el tiempo. Cinco microservicios, API Gateway y bus de eventos en 15 semanas con tres personas (riesgo R-05) no era realista. Un MVP con uno o dos servicios, o un monolito modular que evolucione a microservicios, habría dado una línea base más alcanzable.

Dejar la calidad y la seguridad para el último sprint es un riesgo. El cronograma ubicaba QA en las semanas 13-14; al adelantarlo comprobamos que incorporar pruebas y escaneos desde el primer commit cuesta poco y evita retrabajo. Cada sprint debe tener una definición de terminado que incluya pruebas, Quality Gate y escaneo.

El backlog debe reflejar dependencias. Ninguna de las cinco historias de usuario cubría el registro y la autenticación, aunque todas dependen de ellos. Un backlog con historias habilitadoras, criterios de aceptación y estimaciones habría evitado reordenar el trabajo.

Los requisitos medibles se convierten en pruebas automáticas. Los RNF con cifras (tiempos, usuarios, disponibilidad) fueron los más fáciles de traducir en criterios del pipeline; conviene redactar así todos los requisitos, incluido RNF-03 de seguridad.

Las decisiones de tecnología deben cerrarse y documentarse. La mención de "MongoDB/PostGIS" dejó abierta la capa de datos. Un registro de decisiones de arquitectura (ADR) con la justificación evita ambigüedad al implementar.

La infraestructura también se planea. El plan no incluía entornos ni despliegue; el pipeline y el entorno de prueba deben ser entregables desde la fase de planificación.

6.2 Durante la implementación

1. **La reproducibilidad del build es parte de la calidad.** La corrida #1 falló por no versionar el package-lock.json; las dependencias deben fijarse desde el primer commit.
2. **Las herramientas automáticas encuentran lo que no imaginamos.** ZAP halló un error 500 con un id que ninguna prueba manual había contemplado. Hay que validar tipo y rango de toda entrada en el borde del sistema.
3. **Los frameworks no son seguros por defecto.** Express no envía cabeceras de seguridad y anuncia su tecnología. Una plantilla de servicio ya endurecida evita repetir el mismo hallazgo en cada microservicio.
4. **Diseñar para la prueba acelera todo.** Separar rutas, servicio y repositorio, e inyectar la base de datos, permitió usar PostgreSQL en memoria y lograr cerca de 99 % de cobertura con pruebas que corren en unos 3 segundos.
5. **Una meta que el pipeline verifica se cumple siempre.** El umbral de 80 %, el Quality Gate y el bloqueo por alertas altas convierten los objetivos de calidad en reglas que no dependen de la memoria del equipo.
6. **Medir antes y después da evidencia objetiva.** Ejecutar el mismo pipeline antes y después de las correcciones documentó la mejora (3 → 0 alertas bajas, 2 → 0 code smells, 10 → 0 min de deuda) sin trabajo manual adicional.
7. **Los secretos se generan por entorno.** El secreto JWT por defecto de la primera versión se eliminó; en el pipeline se generan secretos aleatorios en cada despliegue.

7. Plan de mejora continua

El plan sigue el ciclo **Planificar - Hacer - Verificar - Actuar (PDCA)**: cada acción tiene un indicador que el propio pipeline o la plataforma puede medir, una meta y un plazo expresado en sprints de dos semanas. La revisión de avance se hará en cada retrospectiva de sprint.

7.1 Acciones de mejora del proceso y del producto

#	Acción	Indicador	Meta	Plazo
1	Versionar package-lock.json, volver a npm ci y activar Dependabot	Fallos de build por dependencias; vulnerabilidades altas en npm audit	0 y 0	Sprint 1

#	Acción	Indicador	Meta	Plazo
2	Actualizar las acciones de GitHub a versiones compatibles con Node 24 y fijar la versión del runner	Avisos de deprecación por corrida	0 avisos	Sprint 1
3	Crear una plantilla de microservicio endurecida (seguridad, pruebas, Dockerfile y pipeline) para los demás servicios	% de servicios nuevos creados desde la plantilla	100 %	Sprint 2
4	Mantener umbrales de calidad en todos los servicios	Cobertura, Quality Gate y alertas de ZAP	Cobertura ≥ 90 %, QG aprobado, 0 alertas medias/altas en cada push	Continuo
5	Reducir la duración del pipeline con caché de dependencias e imágenes	Duración total (línea base: 5 min 9 s)	≤ 4 min	Sprint 2
6	Escaneo semanal programado (ZAP full scan) y análisis de imagen y dependencias con Trivy	Vulnerabilidades críticas/altas abiertas; tiempo medio de corrección	0 abiertas; corrección ≤ 5 días	Sprint 3
7	Pruebas de carga con k6 en el pipeline para validar RNF-01	Tiempo de respuesta p95 con 1,000 usuarios concurrentes	p95 < 2 s	Sprint 4
8	Refresh tokens con revocación y bloqueo temporal de cuenta tras intentos fallidos	Endpoints de autenticación con pruebas de abuso	100 %	Sprint 3
9	Medir métricas DORA del pipeline	Frecuencia de despliegue, lead time, tasa de fallos de cambios (línea base: 1 de 3 corridas = 33 %) y MTTR	≥ 1 despliegue diario a staging, lead time < 1 día, fallos < 15 %, MTTR < 1 h	Desde el sprint 2
10	Documentación viva: ADR por decisión de arquitectura, OpenAPI publicada y README por servicio	Decisiones de arquitectura documentadas	100 %	Continuo
11	Retrospectiva quincenal con acciones de mejora (PDCA)	Acciones cerradas / acciones acordadas	≥ 80 %	Cada sprint

Tabla 13. Acciones de mejora continua con criterios de medición.

7.2 Propuestas de innovación

Predicción de donaciones con inteligencia artificial. Un modelo de series de tiempo (por ejemplo, Prophet o gradient boosting) entrenado con el historial de cada donante, el día de la semana, la temporada y eventos como campañas navideñas estimará cuántos kilogramos y de qué categorías se donarán en cada zona durante la siguiente semana. Las organizaciones podrán planear su capacidad de recolección y almacenamiento, y la plataforma podrá escalar sus servicios antes de los picos (riesgo R-06).

- **Indicador:** Error porcentual medio (MAPE) menor a 20 % a 7 días; aumento del porcentaje de donaciones asignadas en menos de 24 h.

Asignación inteligente por fecha de caducidad. Un algoritmo de emparejamiento priorizará las donaciones más próximas a caducar y las asignará a la organización compatible más cercana según distancia, capacidad y necesidades declaradas.

- **Indicador:** Reducción de 30 % en kilogramos que caducan sin asignarse.

Optimización de rutas de recolección. Resolver el problema de ruteo de vehículos con OpenStreetMap/OSRM para agrupar recolecciones de varios donantes en una misma ruta.

- **Indicador:** Reducción de 20 % en kilómetros recorridos por kilogramo recolectado.

Registro de donaciones con visión por computadora. Fotografiar la etiqueta del producto para extraer con OCR el nombre, el lote y la fecha de caducidad, y reducir la captura manual en dispositivos móviles (RNF-04).

- **Indicador:** Tiempo de registro de una donación menor a 1 minuto.

Notificaciones por WhatsApp y SMS. Enviar avisos de donaciones compatibles por los canales que las organizaciones ya usan, para mitigar el riesgo de baja adopción (R-02).

- **Indicador:** Tasa de respuesta a notificaciones mayor a 60 %.

7.3 Hoja de ruta

Periodo	Entregables
Sprints 1-2 (octubre 2026)	Acciones 1 a 3 del plan de mejora; servicio de Donaciones (HU-1) creado desde la plantilla, con los mismos umbrales de calidad; API Gateway validando el JWT existente.
Sprints 3-4 (noviembre 2026)	Servicio de Geolocalización con PostGIS y mapa (HU-2); Notificaciones con RabbitMQ (HU-3); escaneos programados, Trivy y endurecimiento de autenticación.
Sprints 5-6 (diciembre 2026)	Seguimiento de estado (HU-4) y Reportes de impacto (HU-5); pruebas de carga con k6; lanzamiento del MVP.
Primer trimestre 2027	Piloto con organizaciones y donantes reales; recolección de datos; prototipo de predicción de donaciones y asignación por caducidad.

Tabla 14. Hoja de ruta propuesta.

8. Conclusiones

La etapa final cumplió los objetivos de implementación, seguridad y calidad: el microservicio de usuarios y autenticación funciona de punta a punta, protege sus rutas con JWT y roles, supera ampliamente el 80 % de cobertura exigido y se entrega mediante un pipeline que prueba, analiza, construye, despliega y escanea cada cambio sin intervención manual.

El valor principal del proceso fue el ciclo de retroalimentación. Las herramientas automáticas detectaron problemas reales —una construcción no reproducible, un error 500 y cabeceras de seguridad ausentes— que se corrigieron y verificaron con el mismo pipeline, dejando evidencia objetiva de la mejora.

Frente a lo planificado, el equipo sacrificó amplitud (un servicio en lugar de cinco) a cambio de profundidad y calidad. Las lecciones aprendidas se tradujeron en un plan de mejora medible y en una hoja de ruta que reutiliza esta base para construir el resto de Alimenta+ y avanzar hacia propuestas de innovación como la predicción de donaciones con inteligencia artificial.

Anexo A. Contenido del repositorio y evidencias

Ruta	Contenido
services/auth-service/	Código del microservicio, Dockerfile y pruebas Jest
.github/workflows/ci-cd.yml	Definición del pipeline de CI/CD

Ruta	Contenido
docker-compose.yml	Entorno de prueba (servicio + PostgreSQL)
scripts/	Pruebas de humo y generadores de reportes de ZAP y SonarQube
reports/antes/	Reportes de la corrida #2: cobertura, JUnit, SonarQube (JSON, capturas) y OWASP ZAP (HTML, JSON)
reports/despues/	Los mismos reportes de la corrida #3, después de las correcciones
docs/	Este informe

Todas las ejecuciones del pipeline, con sus resúmenes y artefactos, pueden consultarse en <https://github.com/Rob-nm/alimenta-plus/actions>.